ARQUITECTURA VOICE AI & ESCALABILIDAD

Guía Maestra de Pruebas

Evaluación de Alucinaciones, Estrés de Concurrencia y Calidad para Agentes de Voz Telefónicos (Call Centers / Cronix SaaS)

Sistema Auditado:	Voice Call Agent (Twilio + Pipecat + Deepgram + LLM)
Destino:	Integración SaaS Cronix / Call Center Concurrente
Objetivo:	Detección de fallos, alucinaciones y límites de carga
Fecha de Emisión:	Octubre 2026
Autor:	Antigravity AI Engineering

1. El Reto Técnico: ¿Por qué Voice AI no se prueba como una API tradicional?

En el desarrollo de software tradicional, herramientas como **k6** o **JMeter** prueban APIs enviando peticiones HTTP (GET/POST) y midiendo milisegundos hasta recibir un código de estado (200 o 500).

Un **Agente de Voz en Tiempo Real** que opera en un call center funciona de forma radicalmente distinta:

- **Streaming Bidireccional Continuo:** No son peticiones aisladas; Twilio Media Streams y WebRTC mantienen WebSockets abiertos transmitiendo paquetes de audio en tiempo real (G.711 μ-law a 8kHz o PCM a 16kHz/24kHz) cada **20 milisegundos** en ambos sentidos.
- **Pipeline en Cascada Multi-Proveedor:** Cada segundo de audio pasa por: *VAD local → Deepgram Flux (STT) → LLM / Pipecat Flows → Deepgram Aura-2 (TTS) → Telefonía*. Un retraso o fallo en cualquiera de las fases rompe la llamada.
- **La Doble Dimensión del Fallo:**
  - **Fallo de Infraestructura (Carga):** Saturación del Event Loop de Python (FastAPI/asyncio), saturación de memoria por búferes de audio, y caídas por Rate-Limits (HTTP 429) de proveedores de IA.
  - **Fallo Cognitivo (Lógica y Alucinaciones):** Cuando hay 50 personas hablando a la vez, ¿el modelo inventa citas? ¿Asume respuestas sin que el cliente las dé? ¿Pierde la máquina de estados si el usuario tose o interrumpe?

> **Principio Clave:** Probar un call center de Voice AI requiere evaluar tanto la **concurrencia de red/audio** (¿se cae el servidor?) como la **fidelidad conversacional** (¿el bot alucina o agenda correctamente?).

2. Catálogo de Herramientas, Funcionalidad y Costos

1. Hamming AI (hamming.ai)

Plataforma Especializada Voice AI

Propósito: Simulación de llamadas telefónicas concurrentes a gran escala para call centers con evaluación automática de alucinaciones.

Cómo funciona: Agentes sintéticos llaman a los números reales de tu central telefónica (Twilio/SIP/WebRTC). Hablan con tu agente mediante voces humanas sintéticas, balbucean, interrumpen (*barge-in*), dudan y cambian de fecha u hora para verificar si el agente de voz resiste la interacción.

Qué fallos detecta: Alucinaciones de citas, saltos de guardrails de seguridad, latencia TTFA (*Time to First Audio*) en percentiles P50, P95 y P99, y tasa de éxito de la tarea (TSR).

Estructura de Costos:

- **Prueba Gratuita:** Sí (créditos de bienvenida de 100 a 250 minutos de simulación al registrarse).
- **Starter / Growth:** Desde **$200 a $500 USD/mes** (incluye 1,000 a 2,500 minutos de llamadas de prueba sintéticas).
- **Enterprise / Call Center:** Precio personalizado según volumen (entre **$0.10 y $0.20 USD por minuto simulado**). Permite conectar tu propia cuenta de Twilio (BYOC) para no pagar sobreprecio de telefonía.

2. Coval (coval.dev)

Simulación Adversarial & Regresión de Voz

Propósito: Evaluación profunda de máquinas de estados y diálogos complejos. Diseñada por ex-líderes de simulación de vehículos autónomos de Waymo.

Cómo funciona: Se conecta directamente a tu arquitectura (nativo con **Pipecat**, Deepgram y OpenAI/OpenRouter). Ejecuta cientos de conversaciones paralelas creando personas con comportamientos adversariales (clientes enfadados, que no cooperan o cambian de opinión bruscamente).

Qué fallos detecta: Errores de transición de nodo en Pipecat Flows, desvíos de contexto, argumentos malformados en tools (como `service_id` inexistentes) y reintentos fallidos.

Estructura de Costos:

- **Developer Sandbox:** Gratuito (simulaciones limitadas de hasta 50-100 conversaciones/mes en entorno de desarrollo).
- **Team Tier:** Desde **$250 a $499 USD/mes** para integración continua en CI/CD y suites de regresión automatizadas.
- **Enterprise:** Cotización anual para call centers con más de 10,000 simulaciones mensuales y soporte SLA.

3. Locust (con Client WebSocket de Audio)

Open-Source / Estrés de Red

Propósito: El verdadero sustituto open-source de k6 para streaming de telefonía. Pruebas de estrés y carga bruta de WebSockets concurrentes.

Cómo funciona: Se programa un usuario virtual en Python que abre una conexión WebSocket con el endpoint `/twilio/stream`, envía el handshake JSON `start` y bombea paquetes de audio G.711 µ-law cada 20 ms, emulando 10, 50 o 200 llamadas activas simultáneamente en el servidor.

Qué fallos detecta: Cuello de botella en el Event Loop de FastAPI (asyncio), fuga de memoria (RAM) por acumulación de búferes de audio, saturación de CPU y límites de sockets del sistema operativo (`ulimit`).

Estructura de Costos:

- **Licencia: 100% GRATUITO y Open-Source** (Licencia MIT).
- **Costo de Infraestructura:** $0 en tu máquina local. En servidores de prueba cloud (AWS EC2 / DigitalOcean), entre **$15 y $40 USD** al mes para generar cientos de clientes concurrentes distribuidos.

4. SIPp (sipp.sourceforge.net)

Open-Source / Telefonía SIP Pura

Propósito: Generador de tráfico de telefonía estándar mundial para pruebas de centrales telefónicas (PBX) y enlaces SIP Trunk.

Cómo funciona: Emite tráfico SIP real y paquetes RTP con audio codificado directamente contra los Gateways de telecomunicaciones de Twilio o la infraestructura del call center.

Qué fallos detecta: Jitter de audio, pérdida de paquetes UDP en telefonía, saturación de troncales y rechazos por concurrencia telefónica.

Estructura de Costos:

- **Licencia: 100% GRATUITO y Open-Source** (Licencia GPL).
- **Costo:** $0 de software. Solo pagas las tarifas de consumo de los minutos de prueba a tu proveedor telefónico (Twilio/Carrier) si la llamada sale a la red pública.

5. DeepEval / Braintrust

Evaluación Funcional de Diálogos & Transcripciones

Propósito: Calificación automatizada de transcripciones mediante LLM-as-a-Judge (ya implementado en este repositorio vía `deepeval-4.2.8`).

Cómo funciona: Tras finalizar las llamadas (sean reales o de prueba), un modelo juez (Gemini Flash / GPT-4o) califica métricas como `invents-nothing`, `natural-spanish` y cumplimiento de herramientas.

Estructura de Costos:

- **Librería Python (DeepEval): 100% GRATUITA** (Open-Source Apache 2.0).
- **Costo de Consumo:** Solo los tokens del modelo juez (usando Gemini Flash directo con `GEMINI_API_KEY` es prácticamente gratuito o centavos por corrida; en OpenRouter cuesta según el volumen evaluado).
- **Dashboard Confident AI:** Opcional, capa gratuita para desarrolladores, planes de equipo desde **$49 USD/mes**.

3. Matriz Comparativa de Capacidades y Costos

Herramienta	Tipo de Prueba	Costo de Entrada	Detección de Alucinaciones	Simulación de Clientes Reales	Estrés de Concurrencia (WebSockets)
Hamming AI	E2E Telefónico + QA	$200 - $500/mes (Free Trial)	Excelente (Automático)	Sí (Voces humanas sintéticas)	Alta (Cientos de llamadas)
Coval	Simulación Diálogo Voice AI	$250 - $499/mes (Free Tier)	Excelente (Adversarial)	Sí (Personas de prueba)	Media / Alta
Locust (Audio WS)	Carga y Estrés de Servidor	**$0 (Open Source)**	No (Solo métricas técnicas)	No (Inyección de audio)	**Muy Alta (Miles de streams)**
SIPp	Carga Telefónica SIP	**$0 (Open Source)**	No (Solo paquetes RTP)	No (Audio pregrabado)	**Masiva (Troncales completas)**
DeepEval	LLM-as-a-Judge (Transcripción)	**$0 (Open Source)**	Excelente (Semántico)	No (Post-llamada)	No (Evaluación analítica)

4. ¿En qué etapas del proyecto usar cada herramienta?

Fase 1: Desarrollo Activo y CI/CD (La fase actual)

Objetivo: Evitar regresiones de código y asegurar que cada commit no rompa la máquina de estados sin gastar presupuesto.

- **Herramientas: Golden Dataset interno + DeepEval** (ya configurado en el repo).
- **Acción:** Correr las pruebas unitarias y conversacionales en loopback local sin tocar telefonía real. Usar el filtro `--only-if-changed` para no gastar créditos innecesarios.
- **Costo estimado: $0 USD** (o menos de $2/mes en tokens de juez).

Fase 2: Conexión con Cronix SaaS & Staging (Pre-producción)

Objetivo: Validar que la API de Cronix aguante la concurrencia y calibrar los límites del servidor antes de salir a clientes reales.

- **Herramientas: Locust (Script de Audio) + Coval (Developer Sandbox)**.
- **Acción:**
    1. Ejecutar un script de **Locust** simulando 20 a 50 conexiones simultáneas a `/twilio/stream` para calibrar el `MAX_CONCURRENT_CALLS` de tu `CallSupervisor` y vigilar el uso de memoria RAM.
    2. Usar **Coval** para lanzar 50 simulaciones de agendamiento y verificar que los turnos se creen bien en la base de datos de Cronix sin que el agente alucine confirmaciones.
- **Costo estimado: $0 a $50 USD** (Coval Free Tier + VPS de prueba).

Fase 3: Salida a Producción & Call Center a Escala (Producción Masiva)

Objetivo: Garantizar que cuando 50 o 100 personas llamen al mismo tiempo a la central telefónica, la calidad no se degrade y las citas sean 100% verídicas.

- **Herramientas: Hamming AI + Monitoreo Sentry/Traces**.
- **Acción:**
    1. Programar campañas de estrés semanal en **Hamming AI** donde agentes sintéticos llamen concurrentemente a las líneas telefónicas reales de Cronix.
    2. Monitorear que el percentil P_{95} de latencia se mantenga bajo los 1,800 ms y que la tasa de resolución supere el 95%.
    3. Si se detecta una alucinación o un fallo en producción, exportar la transcripción y convertirla inmediatamente en un caso del Golden Dataset (`golden_dataset/*.yaml`) para que nunca vuelva a ocurrir.
- **Costo estimado: $250 a $500 USD/mes** (justificado como seguro de calidad para una operación comercial de call center).

5. Checklist Recomendado para Preparar el Call Center

- [ ] **Calibrar el Worker de Python:** Actualmente corres un único proceso Uvicorn. Para un call center debes usar múltiples workers (o contenedores Docker orquestados) detrás de un Load Balancer con Redis para compartir el rate-limiting (`RATE_LIMIT_STORAGE_URI`).
- [ ] **Prueba de fuego de WebSockets con Locust:** Lanzar una prueba de 30 WebSockets concurrentes en staging para verificar en qué segundo Deepgram o tu servidor alcanzan el límite de hilos de audio.
- [ ] **Activar alertamiento de latencia:** Configurar alertas cuando la latencia por turno exceda los 2,000 ms en `var/call_traces.jsonl` para cortar llamadas problemáticas antes de que el usuario cuelgue frustrado.